



Lit - Cheat Sheet

Construye Web Components rápidos y ligeros basados en los estándares de la plataforma web.

Stack: Lit 3.x | TypeScript

Índice

- [1. INTRODUCTION](#) (p. 2)
 - [What is Lit?](#) (p. 2)
 - [Getting Started](#) (p. 2)
- [2. COMPONENTS](#) (p. 2)
 - [Defining](#) (p. 2)
 - [Rendering](#) (p. 2)
 - [Reactive properties](#) (p. 3)
 - [Styles](#) (p. 3)
 - [Lifecycle](#) (p. 3)
 - [Shadow DOM](#) (p. 3)
 - [Events](#) (p. 4)
 - [Decorators](#) (p. 4)
 - [Data Flow \(Comunicación Padre-Hijo\)](#) (p. 4)
- [3. TEMPLATES](#) (p. 5)
 - [Expressions](#) (p. 5)
 - [Conditionals](#) (p. 5)
 - [Lists](#) (p. 5)
 - [Built-in directives](#) (p. 6)
 - [Custom directives](#) (p. 6)
- [4. COMPOSITION](#) (p. 6)
 - [Slots](#) (p. 6)
 - [Controllers](#) (p. 7)
- [5. MANAGING DATA](#) (p. 7)
 - [Context](#) (p. 7)
 - [Tasks](#) (p. 7)
- [6. TOOLS AND WORKFLOWS](#) (p. 8)
 - [Requirements & Development](#) (p. 8)
 - [Testing](#) (p. 8)
 - [Publishing & Production](#) (p. 8)
 - [Starter kits & Adding Lit](#) (p. 8)
- [7. SERVER RENDERING](#)  (p. 9)
 - [Overview](#) (p. 9)
 - [Server usage](#) (p. 9)
 - [Client usage \(Hydration\)](#) (p. 9)
 - [Authoring components](#) (p. 9)
 - [DOM emulation](#) (p. 9)
- [8. FRAMEWORKS](#) (p. 10)
 - [React](#) (p. 10)
- [9. LOCALIZATION \(@lit/localize\)](#) (p. 11)
 - [Overview](#) (p. 11)
 - [Runtime mode](#) (p. 11)
 - [Transform mode](#) (p. 11)
 - [CLI and config](#) (p. 11)
 - [Best practices](#) (p. 11)
- [10. RELATED LIBRARIES](#) (p. 12)
 - [Standalone lit-html](#) (p. 12)
 - [Lit Labs](#)  (p. 12)

1. INTRODUCTION

What is Lit?

 [Leer en lit.dev](https://lit.dev)

Lit es una biblioteca ligera (aprox. 5kb) desarrollada por Google. Proporciona una base reactiva y plantillas declarativas para facilitar la creación de **Web Components** nativos, compatibles con cualquier framework (React, Angular, Vue) o sin ninguno.

Getting Started

 [Leer en lit.dev](https://lit.dev)

Inicializa un proyecto base con TypeScript (recomendado):

```
npm create vite@latest my-lit-app -- --template lit-ts
cd my-lit-app
npm install
npm run dev
```

2. COMPONENTS

Defining

 [Leer en lit.dev](https://lit.dev)

Un componente Lit extiende de `LitElement`. Para registrarlo en el navegador, se usa el decorador `@customElement`.

```
import { LitElement } from 'lit';
import { customElement } from 'lit/decorators.js';

@customElement('simple-greeting')
export class SimpleGreeting extends LitElement { }
```

Rendering

 [Leer en lit.dev](https://lit.dev)

El método `render()` devuelve un literal de plantilla `html`. Se ejecuta automáticamente cuando cambian las propiedades reactivas.

```
import { html } from 'lit';

render() {
  return html`<p>¡Hola Mundo!</p>`;
}
```

Reactive properties

[Leer en lit.dev](#)

- **@property (Públicas):** Forman parte de la API del componente (Atributos HTML).
- **@state (Privadas):** Estado interno del componente.

```
import { property, state } from 'lit/decorators.js';

@property({ type: String, attribute: 'user-name' }) name = 'Anónimo';
@property({ type: Number }) count = 0;
@state() private _isOpen = false;
```

Styles

[Leer en lit.dev](#)

Definidos usando `css` e inyectados en el Shadow DOM, encapsulando el diseño.

```
import { css } from 'lit';

static styles = css`
:host { display: block; color: var(--theme-color, blue); }
`;
```

Lifecycle

[Leer en lit.dev](#)

- **connectedCallback()** : Insertado en el DOM (¡Llamar a `super.connectedCallback()`!).
- **disconnectedCallback()** : Eliminado del DOM (Limpieza).
- **willUpdate()** : Antes del render. Para calcular variables derivadas.
- **firstUpdated()** : Después del primer render. El DOM local ya existe.
- **updated()** : Después de cada renderizado.

Shadow DOM

[Leer en lit.dev](#)

Por defecto, Lit usa Shadow DOM (`this.renderRoot`). Para renderizar en el Light DOM global:

```
protected createRenderRoot() { return this; }
```

Events

[Leer en lit.dev](#)

- **Escuchar:** `@click=${this._handler}`
- **Despachar:** Usa la API nativa `CustomEvent`.

```
private _dispatchClick() {  
  this.dispatchEvent(new CustomEvent('my-event', {  
    detail: { message: 'Hola' },  
    bubbles: true, composed: true // Cruza el Shadow DOM  
  }));  
}
```

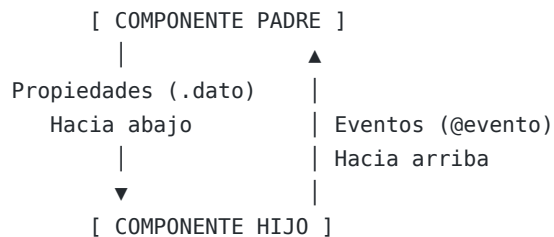
Decorators

[Leer en lit.dev](#)

- `@query('#my-id')` : Referencia al primer elemento en el Shadow DOM.
- `@queryAll('.item')` : Referencia a todos los elementos coincidentes.
- `@queryAsync('#my-id')` : Devuelve una promesa del elemento (útil si está condicionado).

Data Flow (Comunicación Padre-Hijo)

El flujo de datos en Web Components sigue una regla de oro: **Propiedades hacia abajo, Eventos hacia arriba.**



Ejemplo Práctico:

```
// ▲ COMPONENTE PADRE (Superior)  
// Inyecta los datos hacia abajo (.user) y escucha eventos hacia arriba (@user-updated)  
html`<user-card .user=${this.currentUser} @user-updated=${this._onUpdate}></user-card>`  
  
// -----  
  
// ▼ COMPONENTE HIJO (Inferior - user-card)  
// 1. Expone la propiedad pública para recibir los datos del padre  
@property({ type: Object }) user = {};  
  
// 2. En alguna interacción, despacha el evento hacia arriba informando al padre  
this.dispatchEvent(new CustomEvent('user-updated', { detail: this.user }));
```

3. TEMPLATES

Expressions

 [Leer en lit.dev](#)

- **Texto:** `html <p>${this.name}</p>`
- **Atributos:** `html <div id=${this.id}></div>`
- **Booleanos (?):** `html <input ?disabled=${this.isDisabled}>`
- **Propiedades (.):** Pasa objetos. `html <my-list .items=${this.arrayData}></my-list>`
- **Eventos (@):** `html <button @click=${this._handleClick}></button>`

Conditionals

 [Leer en lit.dev](#)

Usa JS estándar (`if` , ternarios) o directivas.

- **Ternarios:** `html ${this.active ? html <b>Sí</b> : html <i>No</i> }`
- **cache()** : Mantiene en memoria componentes inactivos en lugar de destruirlos. Ideal para vistas pesadas.

```
import { cache } from 'lit/directives/cache.js';
html`${cache(this.active ? html`<vista-a></vista-a>` : html`<vista-b></vista-b>`)}`
```

Lists

 [Leer en lit.dev](#)

- **.map()** **nativo:**

```
html`<ul>${this.items.map(item => html`<li>${item}</li>`)}</ul>`
```

- **repeat()** : Renderizado indexado eficiente. Si el array cambia de orden, recicla los nodos DOM existentes basándose en una clave (key) única.

```
import { repeat } from 'lit/directives/repeat.js';

html`<ul>
  ${repeat(this.items, (item) => item.id, (item) => html`<li>${item.name}</li>`)}
</ul>`
```

Built-in directives

[Leer en lit.dev](#)

Las directivas optimizan el renderizado y resuelven tareas comunes. Se importan desde `lit/directives/...`:

- 🎨 **Estilos y Clases**
 - `classMap` : Aplica clases dinámicamente. `class=${classMap({ active: this.isActive })}`
 - `styleMap` : Aplica estilos en línea. `style=${styleMap({ color: this.textColor })}`
- 🔄 **Renderizado y Control de Flujo**
 - `repeat` : Bucle eficiente basado en keys.
 - `map` : Versión simplificada para mapear iterables.
 - `when` : Alternativa declarativa a un if/else ternario.
 - `choose` : Alternativa declarativa a un switch-case.
 - `cache` : Cachea el DOM de las ramas inactivas de un condicional.
 - `keyed` : Fuerza la destrucción y recreación de un nodo cuando su key cambia.
- 🛡️ **Seguridad y DOM Crudo**
 - `unsafeHTML` : Renderiza un string crudo como HTML (¡Cuidado con XSS!).
 - `unsafeSVG` : Renderiza un string crudo como SVG.
- ⚡ **Asincronía e Interacción**
 - `until` : Muestra contenido temporal mientras se resuelve una Promesa.
 - `asyncAppend` / `asyncReplace` : Renderizan valores emitidos por un AsyncIterable.
- 🛠️ **Otras utilidades**
 - `ifDefined` : Si el valor es `undefined`, elimina el atributo HTML por completo.
 - `live` : Fuerza a Lit a ignorar su caché interna y sobrescribir un atributo de un input si fue modificado manualmente por el usuario.
 - `ref` : Obtiene una referencia a un nodo DOM renderizado sin usar `@query`.

Custom directives

[Leer en lit.dev](#)

Extiende `Directive` o `AsyncDirective` para interacciones avanzadas que requieren acceso directo a las partes del DOM antes o durante el ciclo de actualización de Lit.

4. COMPOSITION

Slots

[Leer en lit.dev](#)

Inyecta HTML desde el consumidor (Light DOM) hacia el componente (Shadow DOM).

```
<!-- Consumidor -->
<my-card><h1 slot="title">Título</h1></my-card>
```

```
// Componente
render() {
  return html`<header><slot name="title"></slot></header>`;
}
```

Controllers

[Leer en lit.dev](#)

Extraen lógica de estado o ciclo de vida (ej. Timers, Fetch) fuera del componente visual.

```
import { ReactiveController, ReactiveControllerHost } from 'lit';

export class ClockController implements ReactiveController {
  constructor(private host: ReactiveControllerHost) { host.addController(this); }
  hostConnected() { /* iniciar timer y llamar a this.host.requestUpdate() */ }
}
```

5. MANAGING DATA

Context

[Leer en lit.dev](#)

Evita el "Prop Drilling". Provee datos globalmente para que los hijos los consuman.

```
import { createContext, provide, consume } from '@lit/context';
const userCtx = createContext<User>('user-context');

@provide({ context: userCtx }) @state() user = { name: 'Admin' }; // Padre
@consume({ context: userCtx }) @state() user!: User; // Hijo
```

Tasks

[Leer en lit.dev](#)

El estándar moderno para manejar promesas y llamadas a APIs de forma declarativa.

```
import { Task } from '@lit/task';

class ApiComponent extends LitElement {
  @property() productId = 1;

  private _apiTask = new Task(this, {
    task: async ([id]) => await (await fetch(`/api/item/${id}`)).json(),
    args: () => [this.productId] // Reejecuta si el ID cambia
  });

  render() {
    return this._apiTask.render({
      pending: () => html`<p>Cargando...</p>`,
      complete: (data) => html`<h1>${data.name}</h1>`,
      error: (e) => html`<p>Error: ${e}</p>`
    });
  }
}
```

6. TOOLS AND WORKFLOWS

Requirements & Development

 [Leer en lit.dev](#)

Lit requiere un entorno Node.js y navegadores modernos. Ya que Lit usa módulos ES, necesitas herramientas de desarrollo que soporten resolución nativa de módulos. Para un desarrollo local rápido y con HMR (Hot Module Replacement), la comunidad y el equipo de Lit recomiendan encarecidamente el uso de **Vite** o **Web Dev Server**.

Testing

 [Leer en lit.dev](#)

Los Web Components deben testearse en navegadores reales, no solo en entornos DOM emulados (como JSDOM).

- **Herramienta de Testing:** Web Test Runner (`@web/test-runner`).
- **Librerías de Ayuda:** `@open-wc/testing` (provee funciones como `fixture` y aserciones adaptadas para Shadow DOM).

```
import { fixture, expect, html } from '@open-wc/testing';
import './my-element.js';

it('renders default text', async () => {
  const el = await fixture(html`<my-element></my-element>`);
  expect(el.shadowRoot!.textContent).to.include(';Hola Mundo!');
});
```

Publishing & Production

 [Leer en lit.dev](#)

- **Publicar (Publishing):** Distribuye tus componentes a NPM sin empaquetar, como módulos ES nativos y emite tus archivos `.d.ts` de TypeScript. Deja que el consumidor final decida cómo empaquetarlo.
- **Producción (Production):** Al compilar tu aplicación final para producción, utiliza herramientas (Vite, Rollup) que soporten **minificación** y **tree-shaking** para eliminar el código muerto, asegurando que el bundle de Lit siga pesando lo mínimo indispensable ($\approx 5\text{kb}$).

Starter kits & Adding Lit

 [Leer en lit.dev](#)

- **Añadir Lit a un proyecto existente:** Si ya tienes una aplicación vanilla, Angular o Vue, integrarlo es tan fácil como hacer `npm i lit` y empezar a crear componentes. No requiere configuraciones engorrosas de Webpack gracias al estándar de los módulos ES.
- **Kits de Inicio Oficiales:** Puedes encontrar los repositorios oficiales de plantillas en GitHub (ej. `lit/lit-element-starter-ts`) que incluyen configuración de TypeScript, linting, formateo y testing ya preparados.

7. SERVER RENDERING

Overview

 [Leer en lit.dev](#)

Lit soporta renderizado del lado del servidor (SSR) mediante el paquete `@lit-labs/ssr` (actualmente en fase de laboratorios/experimental). Permite pre-renderizar los Web Components en el backend (Node.js) para mejorar el SEO y reducir el tiempo del *First Contentful Paint* (FCP).

Server usage

 [Leer en lit.dev](#)

En el servidor, no tienes acceso nativo al DOM. Lit provee una función `render` que toma tu plantilla y devuelve un iterable de strings (HTML estático).

```
import { render } from '@lit-labs/ssr';
import { html } from 'lit';
import './my-element.js';

const htmlIterable = render(html`<my-element></my-element>`);
// Unir o streamear el iterable hacia la respuesta HTTP...
```

Client usage (Hydration)

 [Leer en lit.dev](#)

Una vez que el navegador descarga el HTML pre-renderizado, Lit necesita hacerlo interactivo sin reconstruir el DOM. A este proceso se le llama **Hydration**.

- Debes cargar `@lit-labs/ssr-client/lit-element-hydrate-support.js` en el cliente **antes** de definir tus componentes para que Lit sepa que debe "hidratar" en lugar de "reemplazar".

Authoring components

 [Leer en lit.dev](#)

Para que tus componentes sean compatibles con SSR (Isomórficos), debes evitar el uso de APIs exclusivas del navegador (como `window` o `document`) durante la construcción y el `render()`.

- **Regla de oro:** Cualquier código que necesite interactuar con el DOM real (como llamadas a `querySelector`, `addEventListener` al `window`, o librerías externas que asuman que están en un navegador) debe ir dentro de `firstUpdated()` o `connectedCallback()`, ya que estos métodos del ciclo de vida **no** se ejecutan en el servidor, solo en el cliente.

DOM emulation

 [Leer en lit.dev](#)

Lit SSR está diseñado para funcionar sin un DOM completo en Node (lo que lo hace rapidísimo). Sin embargo, si al importar tus módulos de componentes necesitas algunas interfaces globales del DOM, Lit provee un paquete `@lit-labs/dom-shim` que emula las piezas mínimas necesarias (como `HTMLElement`, `customElements`) para que el código no falle (arroje errores de undefined) al evaluarse en Node.js.

8. FRAMEWORKS

React

 [Leer en lit.dev](#)

Aunque los componentes de Lit funcionan perfectamente en Vue, Angular, Svelte o Vanilla JS por ser estándares web nativos, React históricamente ha tenido particularidades a la hora de inyectar propiedades complejas (objetos/arrays) o de escuchar eventos personalizados (CustomEvents).

Para garantizar que un componente Lit se comporte exactamente como un componente React 100% nativo, Lit provee el paquete oficial `@lit/react`.

Este paquete exporta la función `createComponent`, que genera un *Wrapper* de React alrededor de tu Web Component, permitiendo usar *props* clásicas y mapear tus eventos personalizados a los `onEvent` de React.

Ejemplo de Integración:

```
// 1. Instalar la dependencia
// npm i @lit/react

import React from 'react';
import { createComponent } from '@lit/react';
import { SimpleGreeting } from './simple-greeting.js'; // Tu componente Lit

// 2. Crear el componente Wrapper de React
export const SimpleGreetingReact = createComponent({
  tagName: 'simple-greeting',
  elementClass: SimpleGreeting,
  react: React,
  events: {
    // Mapea tu CustomEvent interno a un prop de React (onMyEvent)
    onMyEvent: 'my-event',
  },
});

// 3. Consumirlo en tu JSX
function App() {
  return (
    <SimpleGreetingReact
      name="Ecosistema React"
      onMyEvent={(e) => console.log('El componente Lit disparó el evento:', e.detail)}
    />
  );
}
```

9. LOCALIZATION (@lit/localize)

Overview

 [Leer en lit.dev](#)

Lit posee una herramienta oficial, `@lit/localize`, enfocada exclusivamente en traducir componentes Lit de manera eficiente. Su principal característica es que las traducciones se integran directamente en las plantillas `lit-html`.

Runtime mode

 [Leer en lit.dev](#)

En el modo *runtime* (tiempo de ejecución), las traducciones se cargan dinámicamente (vía `fetch` o importaciones dinámicas JS) mientras la aplicación está corriendo.

- **Pros:** Solo se genera un *build* (paquete) de tu aplicación. Es más fácil de desplegar.
- **Contras:** Hay una pequeña penalización de rendimiento inicial porque el usuario tiene que descargar el idioma antes de que se repinte el componente.

Transform mode

 [Leer en lit.dev](#)

En el modo *transform* (tiempo de compilación), las traducciones se inyectan estáticamente durante el proceso de *build*.

- **Pros:** Es **extremadamente rápido**. No hay penalización de rendimiento (Zero overhead).
- **Contras:** Genera una carpeta o paquete completo distinto por cada idioma (ej. `/es/index.html`, `/en/index.html`), por lo que la gestión en el servidor o CDN es ligeramente más compleja.

CLI and config

 [Leer en lit.dev](#)

Para automatizar la extracción de textos y compilación, utilizas el paquete CLI `@lit/localize-tools`.

1. Se define un archivo `lit-localize.json` indicando tu idioma de origen (ej. `en`) y tus idiomas objetivo (ej. `es`, `fr`).
2. El comando `lit-localize extract` busca en tu código y genera archivos XLIFF (`.xlf`), el formato estándar para agencias de traducción.
3. El comando `lit-localize build` compila esos `.xlf` de vuelta a código TypeScript/JavaScript.

Best practices

 [Leer en lit.dev](#)

- Envuelve todos los strings traducibles en la función `msg()` que proporciona la librería.
- **Usa plantillas dentro de `msg()`** para interpolar variables, en lugar de concatenar cadenas sueltas. El traductor necesita contexto completo de la oración.

```
import { msg, str } from '@lit/localize';

render() {
  // Evitar esto (difícil de traducir)
  // return html`<p>${msg('Bienvenido')} ${this.user}</p>`;

  // Hacer esto (Provee contexto total)
  return html`<p>${msg(str`Bienvenido, ${this.user}`)}</p>`;
}
```

10. RELATED LIBRARIES

Standalone lit-html

 [Leer en lit.dev](https://lit.dev)

Aunque Lit suele usarse para crear Web Components con la clase `LitElement`, puedes usar el motor de plantillas `lit-html` de manera totalmente independiente.

Esto es útil cuando quieres el poder del renderizado rápido y declarativo en una aplicación Vanilla JS **sin la encapsulación ni el ciclo de vida de un Web Component**.

```
// Usar lit-html directamente sin LitElement
import { html, render } from 'lit-html';

// 1. Declarar la plantilla
const myTemplate = (name) => html`<p>Hola, ${name}!</p>`;

// 2. Renderizarla físicamente dentro de cualquier elemento del DOM clásico
const container = document.body;
render(myTemplate('Mundo'), container);
```

Lit Labs

 [Leer en lit.dev](https://lit.dev)

Lit Labs es el área de pruebas donde el equipo de Google desarrolla y estabiliza nuevas funcionalidades antes de integrarlas al core oficial de Lit. Usar paquetes de `@lit-labs/` implica que son experimentales y sus APIs pueden tener *breaking changes* (cambios que rompen compatibilidad).

Algunos de los paquetes de Labs más interesantes incluyen:

- `@lit-labs/virtualizer` : Un componente vital para renderizar listas con miles de elementos, mostrando solo los que caben en pantalla para mantener el rendimiento al máximo (Scroll Virtual).
- `@lit-labs/router` : Un enrutador del lado del cliente diseñado específicamente para componentes Lit, con soporte de rutas anidadas.
- `@lit-labs/ssr` : El paquete para renderizado del lado del servidor (que ya repasamos en la sección 7).
- `@lit-labs/motion` : Directivas súper sencillas para hacer animaciones FLIP que transicionan elementos suavemente cuando se reordenan o mueven por la pantalla.